Bitácora 06 — Cierre técnico

TFM · Predicción de churn en e-commerce (UCI Online Retail II)
Máster Ejecutivo en Big Data & Business Analytics · EUDE Business School
5 de octubre de 2026

Esta bitácora consolida el tramo posterior a la Parte 5 y corrige la Parte 3, que describía XGBoost como «implementado, ejecución pendiente» cuando en realidad se creó y ejecutó después (resultados en el traspaso, sección 12).

1. Revisión del repositorio

La suite de tests inicial pasaba completa (59 tests). Se detectaron los siguientes huecos y desajustes:

- Ausencia de artefacto de modelo serializado (`models/` solo contenía `.gitkeep`).
- MLflow sin implementar (solo declarado como dependencia opcional).
- README y traspaso desactualizados; archivos `*.bak` y un paquete residual (`src/tfm_churn_ecommerce/`).
- Versión de Python sin confirmar y dependencias sin fijar en el plan de despliegue.

2. Modelo final serializado

`scripts/train_final.py` reproduce el protocolo del modelo final y guarda el artefacto para la API:

- Random Forest ajustado **solo con train**.
- Calibración Platt (sigmoid) ajustada **solo con valid**, con el modelo base congelado.
- Umbral operativo: máxima precisión con recall ≥ 0,80, elegido **solo en valid**. Test solo se evalúa.
- Salida en `models/`: `churn_model_1.0.0.joblib` y `churn_model_1.0.0_metadata.json` (features, umbral, métricas, SHA-256 del dataset, versiones y commit).

Split	n	Prev. churn	PR-AUC	ROC-AUC	Precision	Recall	F1	Brier	ECE
train	32.537	0,4934	—	—	—	—	—	—	—
valid	17.076	0,6451	0,8357	0,7735	0,7915	0,8004	0,7959	0,1755	0,0337
test	21.668	0,5870	0,8104	0,7789	0,7495	0,8146	0,7807	0,1863	0,0563

Umbral reproducido en el equipo del autor: **0,598547** (≈ 0,599). La recarga del artefacto reproduce las predicciones (diferencia máxima $4{,}4 \cdot 10^{-16}$).

Matriz de confusión (umbral 0,599)	TN	FP	FN	TP
valid	3.738	2.323	2.199	8.816
test	5.486	3.463	2.358	10.361

Hallazgo importante: el umbral depende de la versión de scikit-learn (1.8.0 → 0,5954; 1.9.x → 0,5985). El despliegue debe instalar desde `uv.lock` y no desde `pyproject.toml`.

3. Calibración y umbral por modelo (test)

Modelo	Sin calibrar (Brier / ECE)	Platt en valid (Brier / ECE)	PR-AUC	ROC-AUC
Logística	0,2034 / 0,1201	0,1886 / 0,0385	0,7852	0,7702
Random Forest	0,2094 / 0,1504	0,1863 / 0,0563	0,8104	0,7789
XGBoost	0,2275 / 0,1866	0,2000 / 0,1025	0,8008	0,7668

Platt por validación cruzada sobre train no transfiere a test (en XGBoost el ROC-AUC cayó por debajo de 0,5); isotonic se probó solo como sensibilidad (RF en test: Brier 0,1856, ECE 0,0566, sin mejora práctica sobre Platt). La prevalencia de churn deriva entre splits (49,3 % → 64,5 % → 58,7 %), lo que explica la mala calibración de los modelos sin recalibrar.

4. Monetary: bruto frente a neto

Regla vigente: **bruto** (las cancelaciones se eliminan antes de calcular las features). El neto refleja mejor el valor del cliente, pero debía compararse antes de congelar la regla, tal como pedía el documento técnico. Se implementó `src/features/returns.py` (devoluciones acumuladas hasta T, sin leakage y con 5 tests) y `scripts/experimento_monetary_neto.py` (3 variantes × 5 semillas, mismo protocolo del modelo final).

Variante	PR-AUC test	ROC-AUC test	Brier	ECE	Umbral medio
A · bruto (actual)	0,8104 ± 0,0006	0,7789 ± 0,0005	0,1866	0,0584	0,5911
B · neto	0,8114 ± 0,0007	0,7794 ± 0,0006	0,1861	0,0554	0,5932
C · bruto + return_ratio	0,8099 ± 0,0003	0,7790 ± 0,0005	0,1862	0,0553	0,5947

- El 40,3 % de las filas cliente+snapshot tienen alguna devolución acumulada; `monetary_net` ≤ 0 solo en el 0,71 %.
- Correlación entre `monetary` y `monetary_net`: 0,9928.
- El churn medio es menor en clientes con devoluciones (0,429 frente a 0,645), probablemente porque devuelven más los clientes más activos.
- **Decisión de trabajo:** mantener el bruto y reportar el neto como análisis de sensibilidad. Adoptar el neto obligaría a reentrenar, recalibrar, fijar un nuevo umbral y rehacer SHAP, el análisis por segmento y la presentación.

5. SHAP y análisis por segmento

SHAP explica el modelo base (imputación → RF) con `TreeExplainer`; Platt solo remapea la probabilidad y no cambia el ranking. Importancia global en una muestra de test:

Variable	Media |SHAP|	Efecto medio
monetary	0,0751	hacia no churn
frequency	0,0749	hacia no churn
recency	0,0602	hacia churn
n_products	0,0309	hacia no churn
month_cos	0,0250	hacia no churn
avg_days_between_purchases	0,0197	hacia churn
month_sin	0,0149	hacia churn
is_post_holiday	0,0038	hacia no churn
n_countries	0,0000	nulo

Segmento wholesale (monetary ≥ p90 de train = 2.736,07), con el modelo congelado y el umbral global:

Segmento (test)	n	Prevalencia	Recall	Precision	ROC-AUC
Wholesale	4.628	0,248	0,333	0,667	0,7945
Resto	17.040	0,679	0,862	0,753	0,7035

No se reentrena: se documenta como limitación y trabajo futuro (decisión D6).

6. API con FastAPI + Pydantic

Archivos `src/api/schemas.py`, `predictor.py` y `main.py`, con `tests/test_api.py` (21 tests).

Endpoint	Descripción
GET /health	ok o degraded según haya modelo cargado.
POST /predict	Probabilidad calibrada, risk_level (HIGH si ≥ umbral), umbral y versión del modelo.
POST /predict?explain=true	Añade top_factors: 3 variables con mayor |SHAP| (valor, contribución y dirección).

- Entrada: `recency`, `frequency`, `monetary`, `n_countries`, `n_products`, `avg_days_between_purchases` (opcional) y `cutoff_date` (opcional).
- Las features estacionales (`month_sin`, `month_cos`, `is_post_holiday`) se derivan de `cutoff_date`.
- Se rechazan campos desconocidos; no se registran payloads ni identificadores. Errores: 422 por validación, 503 sin modelo, 400/500 sin trazas.
- Features, umbral y versión se leen del JSON de metadatos; la ruta del modelo se configura con `CHURN_MODEL_PATH`.
- Verificado con `TestClient` y por HTTP contra el artefacto real: la probabilidad coincide con la del modelo usado directamente y SHAP cumple la propiedad aditiva.
- Límite de alcance: la API recibe features ya calculadas por cliente.

7. Docker

`Dockerfile` (python:3.12-slim, `uv sync --frozen --no-install-project`, usuario sin privilegios, HEALTHCHECK que exige `model_loaded: true`) y `.dockerignore`. La instalación desde el lockfile y la API sobre ese entorno se validaron; **el `docker build` queda por probar en local.**

8. Estado de tests y pendientes

85 tests pasan (59 originales + 21 de API + 5 de devoluciones).

- Probar `docker build` y `docker run` en local.
- Decidir el mecanismo de trazabilidad: MLflow local o mantener `metadata.json` como mecanismo equivalente.
- Confirmar la versión real de Python del entorno (3.12 según `.python-version`; una bitácora mencionó 3.14.3).
- Añadir `httpx` a las dependencias (`uv add httpx`) y sincronizar `uv.lock`.
- Completar `.gitignore`; borrar `*.bak` y `src/tfm_churn_ecommerce/`.
- Opcional: adelgazar la imagen moviendo jupyter, shap y xgboost a un extra `research`.